BATALLA NAVAL 3D — «OPERACIÓN CUBO»

Trabajo Práctico Integrador · Programación 1 / Algoritmo y Estructura de Datos 1

Grupos de 6 integrantes · Python 3.10 o superior · Consigna por etapas

Índice

1. El cubo
2. La flota
3. El arsenal
4. Funcionamiento
5. Búsquedas y recorridos
6. Módulos
7. Condiciones técnicas y restricciones
8. Entregas
9. Documentación
10. Funciones públicas

Desarrollar en equipo (mínimo 5 y máximo 6 integrantes), y en etapas sucesivas, un juego de batalla naval en tres dimensiones que funciona por consola. El tablero clásico de dos dimensiones se reemplaza por un cubo, representado con matrices anidadas.

El sistema se entrega en cinco instancias: una confirmación inicial de grupo, tres entregas de desarrollo y la entrega final. Cada entrega se construye sobre la anterior; lo entregado debe seguir funcionando en las siguientes.

Objetivos

Al finalizar el cuatrimestre el sistema debe:

- Representar el cubo de juego con matrices anidadas y recorrerlo de distintas formas.
- Ubicar una flota de naves de distinto tipo, cada una con su propia regla de ubicación.
- Funcionar por menús y submenús, validando todo lo que se tipea.
- Administrar el estado de la partida con diccionarios, conjuntos y puntos.
- Manejar errores de forma controlada, sin cortarse ante ningún dato inválido.
- Guardar partidas a medio jugar y retomarlas desde el menú principal.
- Contar con pruebas unitarias en cada módulo.
- Resolver la búsqueda de naves con tres métodos distintos.
- Incluir recorridos y efectos recursivos.
- Estar modularizado, documentado y versionado con Git.

> Toda mejora o agregado que respete las restricciones de estructuras y librerías de la sección 7 será considerada.

1. El cubo

El tablero es un cubo de N × N × N celdas, con N configurable y valor por defecto 8. Se representa como lista de listas de listas.

Los tres ejes se llaman **z**, **x** e **y**, y van de 1 a N. Cada celda del cubo es un punto identificado por esos tres valores. Al tipear, un punto se escribe con los tres valores separados por comas, en el orden z, x, y: `3,5,4` es la celda de z 3, x 5, y 4. Un tramo se escribe con los dos puntos unidos por un guión: `3,4,2-3,4,7`. Cada nave ocupa un conjunto de puntos.

Cada celda tiene uno de estos estados: agua sin explorar, nave oculta, agua marcada, impacto, hundido y detectado por sonar. Cada estado debe identificarse en el código con un nombre propio, no con el número suelto.

Visualización

El cubo se muestra por capas de z, con encabezados de x y de y. Además debe poder mostrarse un corte fijando x y otro fijando y.

```
========= CAPA  z = 3 =========
        x1 x2 x3 x4 x5 x6 x7 x8
   y1    ~  ~  ~  ~  ~  ~  ~  ~
   y2    ~  ~  X  ~  ~  ~  ~  ~
   y3    ~  ~  X  ~  o  ~  ~  ~
   y4    ~  ~  ~  ~  ~  ~  ?  ~
   ...

Referencias:  ~ sin explorar   o agua   X impacto
              # hundido        ? detectado
```

2. La flota

Ambos jugadores tienen la misma flota. Cada tipo de nave tiene su propia restricción de ubicación.

Nave	Letra	Celdas	Cant.	Restricción de ubicación	Comportamiento propio
Fragata	F	2	3	Sin restricción.	—
Destructor	D	3	2	Sin restricción.	Al ser impactada revela una celda ocupada del rival.
Submarino	S	3	2	Solo en la mitad inferior de z.	No es alcanzada por el barrido láser.
Crucero	C	4	1	No puede ocupar z = 1 ni z = N.	—
Portaaviones	P	5	1	Solo en la mitad superior de z.	Cada celda requiere dos impactos.
Estación orbital	E	bloque 2×2×2	1	No puede tocar ninguna cara exterior del cubo.	Al hundirse otorga munición extra.

Reglas de ubicación

- Cada nave ocupa una línea recta de celdas contiguas sobre uno de los tres ejes. La estación orbital es la excepción: ocupa un bloque de 2×2×2.
- Ninguna nave puede salir del cubo.
- Entre dos naves debe quedar al menos una celda libre en cualquier dirección, incluidas las diagonales.

3. El arsenal

Cada arma afecta un conjunto de celdas a partir del punto apuntado y tiene una munición inicial fija.

Arma	Letra	Munición	Celdas que afecta
Torpedo	T	ilimitada	Únicamente la celda apuntada.
Misil de racimo	R	3	La celda apuntada y sus seis vecinas ortogonales.
Carga de profundidad	C	2	Toda la recta sobre el eje z que pasa por el punto apuntado.
Sonar	S	4	No produce daño. Revela un plano entero del cubo: se elige un eje y su valor. Con z = 3 recorre todos los x y todos los y de ese plano; con x = 2 recorre todos los z y todos los y.
Barrido láser	L	2	Una recta completa sobre el eje x o sobre el eje y, a elección.
Onda expansiva	O	1	Se propaga en forma recursiva desde el punto, con radio decreciente.
Torpedo guiado	G	1	Si hay un impacto previo sobre una nave no hundida, busca en forma recursiva una celda contigua de esa nave.

La selección del arma y del recorrido debe resolverse con un diccionario que asocie nombre y función. **No se permite una cadena de if / elif para este despacho.**

4. Funcionamiento

El sistema funciona por menús y submenús. Todo se elige de una lista de opciones o se responde a una pregunta; no hay comandos que tipear. El menú vuelve a mostrarse después de cada acción y no se cierra salvo que se elija salir. Un dato inválido nunca corta el programa: se avisa y se vuelve a preguntar.

Hay tres formatos de partida: uno contra uno, uno contra la máquina, y máquina contra máquina. Gana el primero que hunde toda la flota rival.

Menú principal

```
===== OPERACION CUBO =====
1 - Partida uno contra uno
2 - Partida uno contra la maquina
3 - Partida maquina contra maquina
4 - Continuar una partida guardada
5 - Salir
Opcion:
```

Ubicación de la flota

Antes de empezar, cada jugador ubica su flota. Las flotas de la máquina se ubican siempre de forma automática. La nave se elige por su letra y el tramo se indica con los dos extremos unidos por un guión.

```
--- Flota de Jugador 1 ---
1 - Ubicacion manual
2 - Ubicacion automatica
Opcion: 1

Pendientes:  F x3   D x2   S x2   C x1   P x1   E x1

Nave (F/D/S/C/P/E): F
Desde-hasta: 3,5,4-3,5,5
Ubicada.

Nave (F/D/S/C/P/E): D
Desde-hasta: 3,5,6-3,5,8
No se puede ubicar ahi. Intente de nuevo.
```

La ubicación automática coloca la flota completa respetando las mismas reglas.

Turnos

Con las dos flotas ubicadas, el sistema pide confirmación para iniciar y a partir de ahí alterna turnos. En cada turno indica de quién es, pide el arma por su letra y el objetivo, y muestra el resultado del disparo.

```
--- Turno de Jugador 1 ---
1 - Disparar
2 - Ver el cubo
3 - Guardar y salir
Opcion: 1

Arma (T/R/C/S/L/O/G): T
Objetivo: 4,2,7
Impacto. Destructor averiado (1 de 3).

--- Turno de la maquina ---
Dispara a 6,3,1. Agua.

--- Turno de Jugador 1 ---
Arma (T/R/C/S/L/O/G): S
Eje (z/x/y): z
Valor: 3
Sonar sobre el plano z = 3. 2 contactos detectados.
```

Guardado

Desde cualquier turno se puede guardar la partida a medio jugar y salir. La opción 3 del menú principal lista las partidas guardadas y permite retomar una de ellas en el punto exacto en que se dejó.

5. Búsquedas y recorridos

La localización de una nave en el cubo debe resolverse con **tres métodos de búsqueda distintos**:

- Un método por recorrido lineal, con ciclos `for` anidados y sin estructura auxiliar.
- Un método optimizado, cuya estrategia decide y justifica el propio grupo.
- Un método recursivo.

La implementación de cada uno es libre. Los tres devuelven el resultado más un diccionario con las métricas de la corrida:

```
{"comparaciones": 128, "tiempo_ms": 0.42, "profundidad_max": 4}
```

Además, el sistema ofrece al menos tres recorridos del cubo seleccionables desde el menú (por ejemplo: por capas de z, en espiral, por octantes, o propagando desde un impacto). Al menos dos de esos recorridos deben ser recursivos.

6. Módulos

El sistema se organiza en cinco módulos de dominio más un integrador. La asignación de personas la decide el grupo y puede cambiar de una entrega a otra; el rol de integrador rota y nadie puede ocuparlo en dos entregas consecutivas.

Ningún módulo de dominio usa `print` ni `input`: reciben datos y devuelven datos. Toda la entrada y salida vive en el integrador.

Módulo	Responsabilidad
tablero.py	El cubo: creación, validación de puntos, lectura y escritura de celdas, capas y cortes, operaciones de matrices, dibujo.

Módulo	Responsabilidad
flota.py	Catálogo de naves, ubicación manual y automática, reglas de ubicación, estado de cada nave.
radar.py	Los tres métodos de búsqueda, sonar por planos, recorridos del cubo, mapa de probabilidad, medición.
armamento.py	Catálogo de armas, celdas que afecta cada una, aplicación del disparo, munición.
registro.py	Historial de la partida, estadísticas, guardado y carga en JSON, listado de partidas, ranking.
partida.py / main.py	Integrador: menús y submenús, ciclo de turnos, los tres formatos de partida, entrada y salida, captura de excepciones.

Alcance de cada módulo por entrega

Módulo	Entrega 1	Entrega 2	Entrega 3
tablero.py	• Cubo y validación de puntos • Dibujo de un plano de z • Operaciones de matrices	• Planos fijando x y fijando y • Vecinos de una celda	• Recorridos alternativos del cubo
flota.py	• Catálogo de naves • Ubicación manual y automática • Reglas de ubicación	• Impactos • Naves averiadas y hundidas	• Datos de la flota para guardar y reponer la partida
radar.py	• Búsqueda lineal con métricas	• Búsqueda optimizada • Sonar por planos	• Búsqueda recursiva • Recorridos recursivos • Mapa de probabilidad
armamento.py	• Catálogo de armas • Torpedo	• Racimo, carga de profundidad y barrido láser • Munición	• Onda expansiva • Torpedo guiado
registro.py	• Historial de la partida en memoria	• Estadísticas • Resumen al terminar	• Guardado y carga en JSON • Listado de partidas • Exportación y ranking
partida.py	• Menú principal • Submenú de ubicación • Dibujo del estado	• Partida uno contra uno y contra la máquina • Turnos y victoria • Excepciones propias	• Partida máquina contra máquina • Continuar partida guardada • Guardar y salir • Máquina con dos niveles

7. Condiciones técnicas y restricciones

El programa debe

- Estar modularizado, con funciones documentadas (qué recibe, qué devuelve, qué excepciones lanza).
- Validar con expresiones regulares los datos que se tipean: puntos, opciones de menú y nombres de partida.
- Manejar excepciones propias, con clase base y `try / except / else / finally`.
- Tener pruebas unitarias con `pytest` en cada módulo.
- No cortarse nunca por una excepción no capturada.

- Ejecutarse en cualquier computadora sin instalar nada además de lo declarado en `requirements.txt`.
- Usar rutas relativas únicamente. Ejemplo correcto: `datos/partidas/partida.json`.
- Guardar los datos en archivos, no solo en memoria.
- Versionarse con Git: repositorio con los seis integrantes, ramas de trabajo y subida a GitHub o GitLab.

No se permite

- Bases de datos ni interfaces gráficas.
- Programación orientada a objetos, salvo las clases necesarias para las excepciones propias.
- Librerías externas no vistas en clase. Se admite la biblioteca estándar y pytest.
- Reemplazar el cubo de matrices anidadas por otra estructura.
- Resolver el despacho de opciones, armas o recorridos con cadenas de if / elif.

8. Entregas

Instancia	Fecha	Alcance
Entrega 0	24/08	Confirmación del grupo, repositorio creado con los seis integrantes y README con el nombre del proyecto. Sin código.
Entrega 1	28/09	Ver la columna Entrega 1 de la tabla de la sección 6.
Entrega 2	19/10	Ver la columna Entrega 2 de la tabla de la sección 6.
Entrega 3	16/11	Ver la columna Entrega 3 de la tabla de la sección 6.
Entrega final	28/11 y 30/11	El 28/11 envían por correo el código fuente completo y el enlace al repositorio público. El 30/11 se revisa el trabajo en clase y se responden consultas sobre cualquier módulo.

Cada entrega debe mantener funcionando lo entregado anteriormente.

9. Documentación

- `README.md` con instalación, ejecución y explicación de los menús.
- Docstring en toda función pública.
- `docs/bitacora.md` con el registro, por entrega, de quién trabajó en qué módulo.

10. Funciones públicas

Las siguientes funciones deben ser públicas e importables desde afuera de su módulo, con estos nombres. El resto de los nombres del proyecto es libre.

partida.py

Función	Recibe / Devuelve
nueva_partida_1v1	configuración → estado inicial de una partida de dos jugadores.
nueva_partida_vs_maquina	configuración, dificultad → estado inicial de una partida contra la máquina.
nueva_partida_maquina_vs_maquina	configuración, dificultades → estado inicial de una partida entre dos máquinas.

Función	Recibe / Devuelve
ejecutar_turno	estado, jugada → estado actualizado.
turno_maquina	estado → estado actualizado, jugado por la máquina.
hay_ganador	estado → ganador, o ninguno.

flota.py

Función	Recibe / Devuelve
ubicar_nave	cubo, flota, nave, punto desde, punto hasta → cubo y flota actualizados, o excepción.
ubicacion_automatica	cubo, catálogo, semilla → flota ubicada.

registro.py

Función	Recibe / Devuelve
guardar_partida	estado, nombre → archivo escrito.
cargar_partida	nombre → estado, o excepción.
listar_partidas	→ nombres de las partidas guardadas.